Assignment 4: System Programs and Operating-System Structures

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Deadline: two weeks from the date of issue — there is no automatic late penalty; if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).
Answer every question. Show all working for Numerical and Design questions — a bare final answer without the derivation earns partial credit only.
Reading: Silberschatz et al., *Operating System Concepts*, Ch. 2 §2.5–2.8, pp. 66–84 [1]; Stallings, *Operating Systems: Internals and Design Principles*, Ch. 2 §2.3 (p. 64) and Ch. 4 §4.3 (p. 179) [2].

4.1 Conceptual

Q1. System programs are what a user actually sees of the operating system [1].

(a) Classify each of the following into the six system-program categories: `mv`, `df`, `sed`, `as`, the loader, `ftp`. State which category none of them belongs to that a web browser would fall into, and why it is not part of the OS proper.

(b) `who` is a status-information *program*. Name the system-*call* category it most plausibly uses, and explain in two sentences why the two taxonomies differ.

(c) Compare the two designs for a command interpreter (built-in commands vs. commands as system programs) in terms of interpreter size and how a new command is added. A student proposes implementing `exit` (terminate the shell) as a separate program. Explain precisely why it would not work, using the same reasoning the lecture used for `cd`.

Q2. Mechanism, policy, and isolation.

(a) Define mechanism and policy. For each of the following, identify the mechanism and the policy: (i) a disk-scheduling routine that reorders pending requests according to a table of priorities; (ii) the kernel's check of a user's password at login; (iii) a hypervisor that divides CPU time among three virtual machines.

(b) Explain why "layered" does not mean "the upper layers run in user mode," and why loadable kernel modules do not isolate drivers. In each case state the blast radius of the lecture's buggy network driver.

(c) Windows NT moved services from user-level servers back into the kernel. State one thing it gained and one thing it lost, in terms of the lecture's trade-off.

Concepts That Click

4.2 Numerical

Q3. Use the lecture's counting model: each message (or system call) is one trap, and each trap costs 2 boundary crossings (entry + return). A request that passes through a chain of k user-level servers and back costs $2k$ message traps.

(a) A microkernel application reads a file block; the request goes to the file server, which asks the disk-driver server, which asks a block-cache server, and replies return along the same chain. Give k, the number of traps, and the number of crossings. Compare with the monolithic cost.

(b) Each crossing costs 0.4 μs and each message also copies a 64-byte header between address spaces at 0.1 μs per message. Compute the per-request overhead for (a) and for the monolithic design (one trap, no message copy).

(c) A workload issues 250,000 such reads per second. Compute the CPU time per second spent on overhead in each design, and express each as a percentage of one CPU.

Q4. Layer traversal. A layered OS has layers numbered 0 (hardware) to 5 (user interface), and a request at layer i calls each layer below it exactly once on the way down and returns through each on the way up.

(a) A request enters at layer 5 and needs the hardware. How many inter-layer calls (downward) and returns (upward) occur?

(b) Each inter-layer call or return costs 50 ns, and the useful work at the hardware level costs 2 μs. What fraction of the request's time is layering overhead?

(c) The designers merge layers 2, 3, and 4 into one "fewer, more functional" layer. Recompute (a) and (b), and state the percentage reduction in overhead.

4.3 Design

Q5. Order the layers [1]. A small layered OS has five components: CPU scheduler (S), memory manager (M), backing-store driver (D), console driver (C), and file system (F). The calls are: F calls D and M; M calls D; D calls S (to block while waiting for I/O); C calls S; F calls C (to print errors).

(a) Using the rule "if A calls B, B is in a lower layer," derive a valid layer ordering, from lowest to highest, and justify each placement.

(b) Now add the lecture's large-system complication: S's tables may be swapped out, so S calls D. Show that no valid ordering exists, naming the cycle.

(c) Propose one design change that breaks the cycle (for example, pinning the scheduler's tables in memory), and give the resulting ordering.

Q6. Choose a structure. For each system below, choose monolithic, layered, microkernel, modular, or virtual machine (a hybrid is allowed if justified). For each, state where the risky component runs, its blast radius if it fails, and the performance price you accept.

(a) A hospital infusion-pump controller with a third-party Bluetooth logging driver of unknown quality.

Concepts That Click

(b) A university lab that must test students' experimental kernels on shared machines without disrupting other users.

(c) A desktop OS that must support thousands of hardware devices whose drivers are installed after the OS ships.

(d) A web-hosting company consolidating forty lightly loaded servers, each running a different OS version, onto four physical machines.

References

[1] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley and Sons, 2018.

[2] William Stallings. *Operating Systems: Internals and Design Principles*. Prentice Hall, 2001.

Concepts That Click